Intro

Domain of EPART

Pattern recognition system:

- input
- sensing
- segmentation
- feature extraction
- classification (missing features, use of context information)
- post-processing (risk-estimation)
- decision

EPART interests in classification, post-processing and decision

Design cycle

1. start
2. collect data
3. choose features (using domain knowledge)
4. choose model
5. train classifier (problem of overfitting)
6. evaluate classifier

Nearest Neighbour Classification

Template Matching

- each class represented by one template (prototype)
- classified objects are templates with added noise
- comparing objects to templates by maximum corelation or minimum error

Minimum-distance classification

- **feature vector** in place of template:
  - $t_1, t_2, ..., t_L$
  - L - number of classes in problem
- classification:
  - compute feature vector of object x
  - then we compute L distances between x and each template
  - finaly we find minimum distance between x and class templates

Distance functions

- usualy written as $d(x, y)$
- must satisfy 4 axioms:
  - **reflexivity** - if $d(x, y) = 0$ then $x = y$
  - **non-negativity** - $d(x, y) \geq 0$
  - **symmetry** - $d(x, y) = d(y, x)$
  - **triangle inequality** - $d(x, y) + d(y, z) \geq d(x, z)$

1-NN

- nearest neighbour classification
- we try to find class that is nearest to our classified object
- only one nearest neighbour decides about the classification result

Training set

- training set: $T = \{\langle x^k, y^k \rangle, k = 1, 2, ..., N\}$
- pairs of feature vectors x^k and their class labels y^k
- we find minimal distance between our classified object and feature vectors from training data

Advantages

- direct use of training set
- low probability of error
- simple implementation

Disadvantages

- large computational complexity for large N
- sensitive to misclassified samples
- random behaviour at class boundaries

k-NN

- majority voting of k neares neighbours
- we get a set of neighbours T^k from training set T

$$T^k = \{\langle x^v, y^v \rangle, v = 1, 2, ..., k\}$$

- we take a subset of T^k which contains elements of class y ($T^{k,y}$)

- membership function $F^{y(x)}$ - we count how many of k nearest neighbours of class x belongs to class y
- problem - we need k both to be:
  - large - to minimize misclassification
  - small - to seperate classes strongly
- we use training data directly so classifier needs to store all training vectors and their class labels:
  - N - number of samples
  - D - number of features
  - we need memory of $N \times D + N = N(D + 1)$
  - k parameter is irrelevent

Data preprocessing

- in NNC it's important that all features should be equally important
- first step of designing classifier: learn the data
  - detect outliers
  - make decision if normalization is needed

Normalization

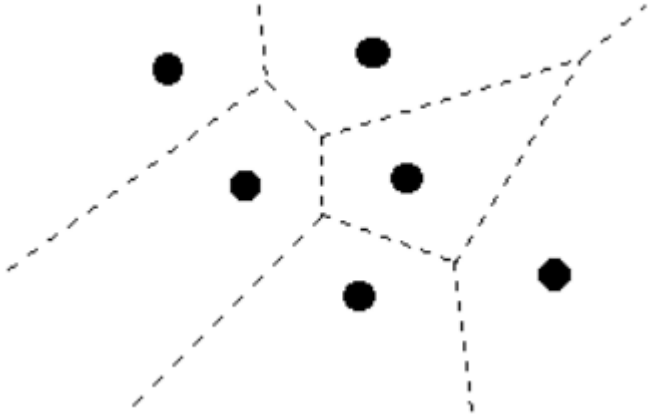
- usually simple operations will do (mean and variance of features, min-max values, plotting histogram)
- in many cases of normalization, scaling the input data will do
  - if you normalize training set, data to classify should be normalized in the exact same way
- normalizing data according to their distribution have some pros:
  - features become comparable (one does not dominate over another)
  - numeric stability
  - often improves performance of classifiers, such as (k-NN, k-means, SVM)
- but it has flawes:
  - some features **are** more important than others
  - scaling is sensitive to outliers
  - again, you have to scale test and inputed data

HNSW

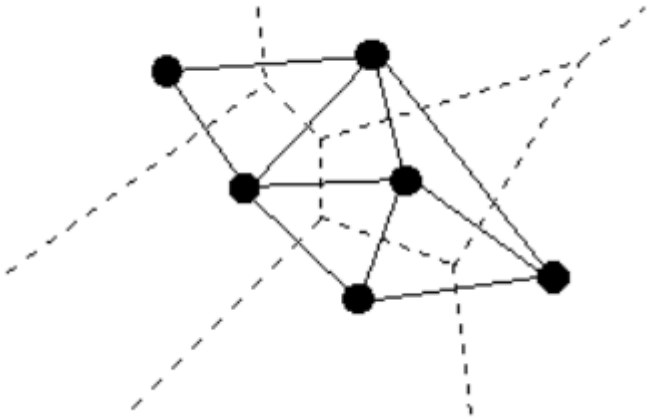
- multi-level map
- top layer has the fewest points
- botton has all points
- we find nearest neighbour to the query, then we go layer down
- you need to build it first, and it's not always 100% correct
- advantages:
  - at first you jump very far
  - then zoom in precisely
  - logarithmic time instead of linear

Voronoi diagram

- for given set of nodes, you divide space into regions in which all points are closer to region's node than any other node



- **Delaunay triangulation** - if two Voronoi regions have common edge, you connect the nodes of those regions
  - they become **Voronoi/Delaunay neighbours**:



Condensing algorithm

1. Compute Delaunay triangulation for training set
2. For each node, mark, if all it's Delaunay neighbours belong to the same class as the node considered
3. Remove all marked nodes (space for chosen node expands). Remaining nodes comprise condensed training set

Editing algorithm

1. Build proximity graph of the training set
2. For each node:
   - Find all it's neighbours
   - Determine most frequent class in the neighbourhood
   - Mark node if it belongs to other class
3. Remove marked nodes

Optimal Bayes Classification

Definition

- classification without measurements
- we base just on probability:

$$c_1 \text{ if } P(c_1) > P(c_2)$$
$$c_2 \text{ otherwise}$$

- P_1, P_2 - *a priori* probabilities - our beliefs about the classes **before** seeing the data
- classifier always makes the same decision, depending only on *priori* probabilities
- classification error: $P(e) = \min\{P(c_1), P(c_2)\}$

Probability density function (pdf)

- gęstość prawdopodobieństwa
- we take continuous variable X
- distribution of x values is described by probability density function $p(x)$
- for each interval $\langle x_1, x_2 \rangle$ we calculate probability of X belonging to interval from equation:

$$P(X \in [x_1, x_2]) = \int_{x_1}^{x_2} p(x)\,\mathrm{d}x$$

- distribution funtion:

$$F(x) = P(X < x) = \int_{-\infty}^{x} p(x)\,\mathrm{d}x$$

Class conditional pdf

- $p(x \mid c_i)$
- distribution of feature values for objects of class c_i

Classification using pdf

- we seek probability that object belongs to c_i under condition that feature x was measured:

$$P(c_i \mid x) = \frac{p(x|c_i)P(c_i)}{p(x)}$$

  - $P(c_i \mid x)$ - *posteriori* probability (after observing x)
  - $p(x \mid c_i)$ - how typical is x for class c_i
  - $P(c_i)$ - *prior* probability
  - $p(x)$ - normalising factor for all classes
- object belongs to:

$$c_1 \text{ if } P(c_1 \mid x) > P(c_2 \mid x)$$
$$c_2 \text{ otherwise}$$

- since $p(x)$ is just scaling factor,

$$c_1 \text{ if } p(x|c_1)P(c_1) > p(x|c_2)P(c_2)$$
$$c_2 \text{ otherwise}$$

Loss and Risk

- loss function: $\lambda(\alpha_i \mid c_j)$
  - α_i - decision

  - $i = 1, ..., C$
  - C - amount of decisions
  - c_j - correct class
- expected loss of decision α_i is

$$R(\alpha_i \mid x) = \sum_{j=1}^{C} \lambda(\alpha_i \mid c_j) P(c_j \mid x)$$

Discriminant functions

- for every clas c_i we define discriminant function $g_i(x)$
- dividing feature space into decision regions
- we assign x to c_i if $g_i(x) > g_j(x)$ for all $j \neq i$
- usually for Bayes $g_i(x) = R(\alpha_i \mid x)$ (discriminant function is a function of expected loss of decision)
- boundry between regions is where probability of 2 classes is the same

Multivariate normal distribution

- for multiple dimenshions, variance is replace by **covariance matrix $\mathbf{\Sigma}$**
- diagonal elements are each feature's variance
- off-diagonal elements represent inter-feature covariances

Bayes decision

- for Bayes classifiers with 2D normal distributions and equal priors, key factor is **covariance matrices**
  - if matrices are equal, we have **linear** decision boundry
  - otherwise, we have **non-linear** decision boundry
- in practice, if shapes are equal, just moved in space, decision boundry is **linear**
- when we analize *a priori* probabilities:
  - probabilities are set in order, which area grew the most, and which shrunk
- memory complexity:
  - C - number of classes
  - D - number of features
  - we need **mean** and **variance** for every class and feature plus class priors
  - that gives us $C \times 2D + C = C(2D + 1)$

Estimating pdf with Parzen window

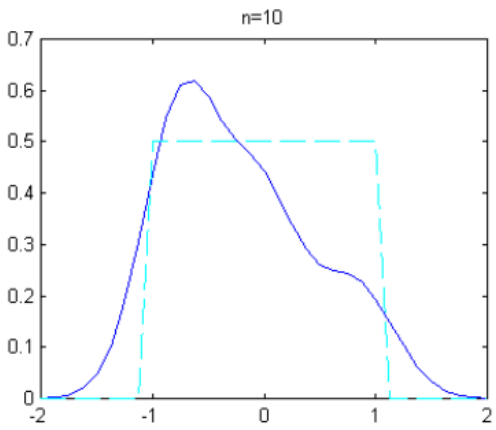
- general idea - "building" unknown dendity from the partial densities of training samples
- **window function** - $\varphi(u)$
  - must be valid density function (non-negative, integral over all domain must be equal to 1)
  - for example simple Gaussian-like window function:

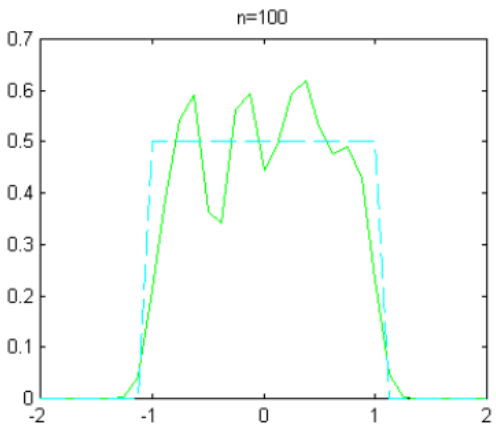
$$\varphi(u) = \frac{1}{\sqrt{2\pi}} e^{-\frac{u^2}{2}}$$

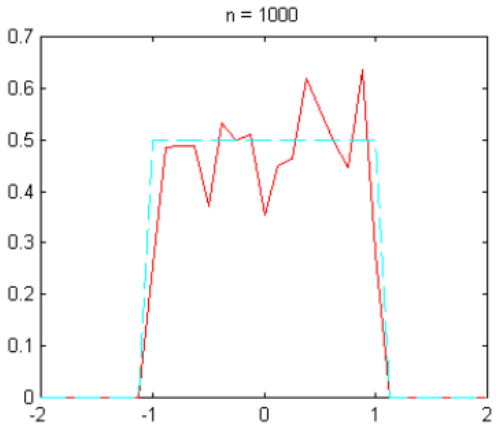
- we often introduce h parameter to scale by number of samples
- pdf estimate is:

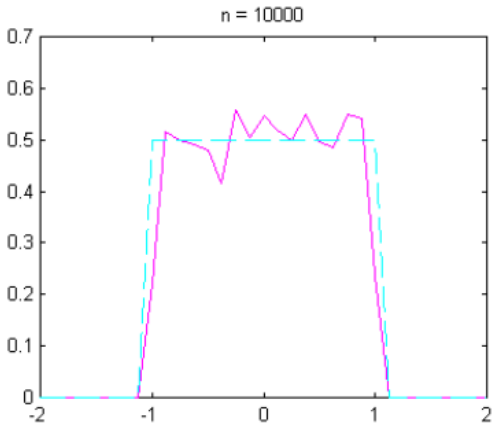
$$p_{n(x)} = \frac{1}{n} \sum_{i=1}^{n} \frac{1}{h_n} \varphi\left(\frac{x - x_i}{h_n}\right)$$

- window with different amount of samples:



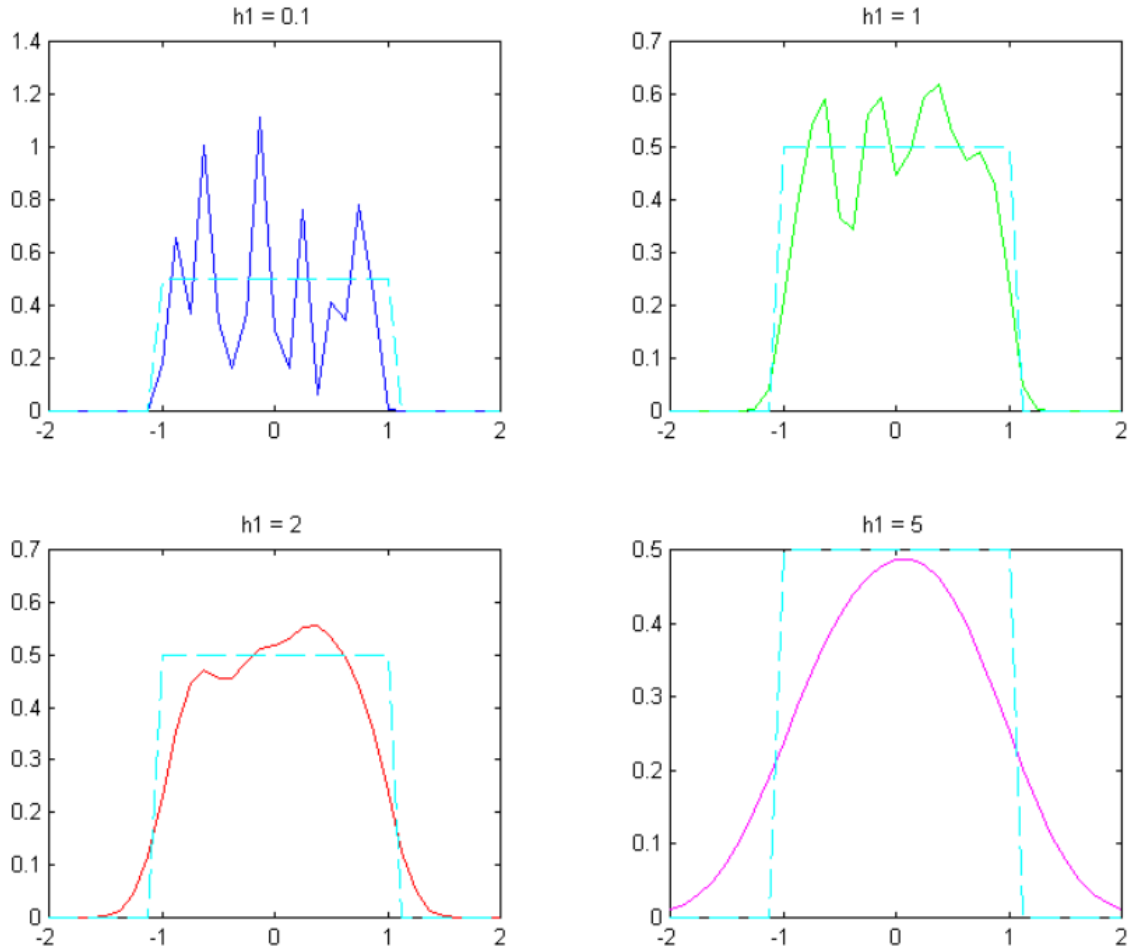






- widths are actually heights (xD):

Different Parzen window *widths*



- memory:
  - for each class classifier must store samples used to create that density ($N \times D$)
  - class labels for samples (N)
  - h parameter
  - final storing: $N(D+1)+1$

Covariance matrix

$$\Sigma = \begin{pmatrix} \sigma_x^2 & \sigma_{xy} \\ \sigma_{xy} & \sigma_y^2 \end{pmatrix}$$

- values σ_x^2 i σ_y^2 - how countour is spread in coresponding axises (bigger value - more spread in that diretion)
- value σ_{xy} - in which direction ellipse is tilted:
  - positive - to the right (/)
  - negative - to the left (\)
  - clockwise - positive
  - also, big value - stronger tilt
  - small value - weaker tilt

Linear Discriminant Functions

Why?

- computing probability density is difficult and often incorrect
- we try to calculate parameters of **discriminant function**
- much simpler classifier at the cost of insignificant decrease of classification quality
- simple formula:

$$g(x) = w^T x + w_0$$

  - w - weight vector (that we try to calculate)
  - x - input vector
  - w_0 - bias vector
- augmented feature vector:

$$a^T = [w_0 w_1 ... w_d] = [w_0 w^T]$$

- discriminant function:

$$g(x) = \sum_{i=0}^{d} w_i w_x = a^T y$$

- in general case we have c discriminant functions - one for each class
- memory needs for one classifier:
  - D - number of features
  - C - number of classes
  - for one classifier we need $D + 1$ memory (all features + bias)
  - all memory needs depends on solution type (one v all, two v all etc.)

Linearly Seperable Case

- if exists such vector a that all training samples can be classified correctly, the samples are *linearly seperable*
- in that case for every y_i belonging to class c_1 we have $a^T y_i > 0$
  - and for every y_j belonging to c_2 we have $a^T y_j < 0$
- we try to maximize distance between samples and seperating plane, to hope that vector will classify new samples better

Gradient methods

- we take criterion function $J(a)$
- we start at selected a_{start}
- in each step we calculate gratient vector $\nabla J(a)$ and move the solution in the directon of the steepest descent:

$$a(k+1) = a(k) - \eta(k) \nabla J(a(k))$$

  - $\eta(k)$ - learing rate
  - we usually try to decraese value of learing rate (for example proportionally to $\frac{1}{k}$)

More than two classes

One vs many/rest

- if we have $c > 2$ classes, we can prepare c linear classifiers
- **one vs. rest** approach

- rarely one class can be separated by hyperplane
- we have very unbalanced sizes of positive and negative samples

One vs one

- we prepare classifiers for every pair of classes(
- better predictions, but many more classifiers $\frac{c(c-1)}{2}$)
- result is read by looking which class got the most votes
- the problem is that each classifier is trained on parts of training set (2 classes of c), but it votes on every classification

Support vectors

- extension of linear discriminant functions
- we demand such a separating plane that all training vectors are not closer to plane than some distance $\frac{d}{2}$
- such a separating plane is determined by few vectors - **support vectors**

Linear Classifier size

Dimensionality Reduction

Problem

- often we have vectors with many many features
- we usually can't asses which features are more important and which less
- also features are usually not independent
- but large dimenshionality urges us to have more training data
- computational complexity also radically grows with high amount of dimenshions

PCA

- finding new coordinate axes in which data has the biggest variance
- ignores class labels
- usefull for data compression and retaining maxiumum information

FLD

- *Fisher's Linear Discriminant*
- finding projection that best seperates the classes
- discrimination is the priority

Clustering

Basics

- unsupervised learning - we have data without labels
- **cluster** - a set of similar objects grouped together
- goal - high similarity within a cluster, but low between clusters

Clustering quality

- **silhouette coefficient** - for i element:

$$s(i) = \frac{b(i) - a(i)}{\max(a(i), b(i))}$$

  - $a(i)$ - mean distance to other samples in the same cluster
  - $b(i)$ - minimum mean distance to samples in other clusters
  - value close to 1 - element well assigned
  - value close to 0 - lies on the boundry between clusters
  - negative - missasigned

Clustering method

- **k-means algorithm**:
  1. choose k initial centroids
  2. assign each point to the nearest centroid
  3. recompute each centroid as the mean of it's assigned points
  4. if any centroid changed, go back to step 2
- problems:
  - bad initialization
  - empty clusters
  - result depends on the metric